

Reporte del proyecto: The Uno

Juego de UNO para 3 jugadores en C# con base de datos

Integrantes

Tonatiuh Charles Pérez Minssart

Héctor Rodrigo Benítez Gama

Alejandro Hernández Hernández

Interfaces Gráficas con Aplicaciones, 223602

Profesor: Alberto Salvador Núñez Varela

11/10/2026

1. Descripción general del proyecto

The Uno es un juego de UNO para 3 jugadores, uno por cada integrante del equipo, que desarrollamos en C# con el framework Avalonia y el patrón MVVM. El juego corre en una sola computadora: una misma persona controla a los tres jugadores por turnos desde la misma pantalla. Las partidas, el historial de victorias y derrotas y todos los movimientos se guardan en una base de datos SQLite por medio de una API REST escrita en Python con FastAPI.

Seguimos las reglas oficiales de Mattel: cada jugador recibe 7 cartas de una baraja de 108; se puede tirar una carta del mismo color, número o símbolo, o un comodín; el Salto hace perder el turno al siguiente jugador; la Reversa invierte el sentido del juego (en 3 jugadores no funciona como Salto); el +2 y el +4 obligan al siguiente jugador a robar y perder su turno, sin acumulación; el +4 solo se puede tirar si el jugador no tiene ninguna carta del color activo; y quien no canta UNO al quedarse con una carta recibe 2 cartas de penalización.

1.1 Arquitectura

Dividimos la solución en tres proyectos, uno por integrante, para que cada quien pudiera trabajar en su parte sin depender del avance de los demás. Los proyectos se comunican por medio de contratos (interfaces) definidos en el Core y de eventos de C#:

- **TheUno.Core:** el motor de juego en C# puro. Contiene los modelos (carta, jugador y baraja), las reglas, la máquina de turnos y los eventos que avisan cuando se tira o se roba una carta, cuando cambia el turno, cuando hay una penalización y cuando termina la partida. No depende de la interfaz ni de la base de datos.
- **TheUno.UI:** la interfaz gráfica en Avalonia con estética de 8 bits. Los ViewModels escuchan los eventos del motor y actualizan la pantalla; las vistas solo muestran el estado y envían las acciones del usuario por medio de comandos.
- **TheUno.Infrastructure:** la persistencia. Incluye la API en Python (FastAPI + SQLite), los repositorios que la consumen desde C# y la versión con Entity Framework Core (entidades, migraciones y repositorios) con la que se diseñó la base de datos.

Al iniciar una partida, la interfaz la registra en la API con los tres jugadores; el motor envía cada movimiento al log y, al terminar, se guardan el ganador, el número de turnos y la duración. Si la API no está abierta, el juego funciona igual, pero sin guardar la partida.

1.2 Tecnologías utilizadas

Tecnología	Uso en el proyecto
C# y .NET 10	Lenguaje y plataforma de toda la solución.
Avalonia 12 y CommunityToolkit.Mvvm	Interfaz gráfica multiplataforma con el patrón MVVM (propiedades observables y comandos).
NAudio / afplay	Música de fondo en Windows (NAudio) y en macOS (afplay).

Tecnología	Uso en el proyecto
Python, FastAPI y SQLite	API REST que guarda jugadores, partidas y movimientos en la base de datos.
Entity Framework Core	Diseño de la base de datos: entidades, migraciones y semilla de jugadores.
Git y GitHub	Control de versiones, una rama por integrante y pull requests hacia main.

1.3 Capturas del juego



Figura 1. *Menú principal con los botones para iniciar el juego y ver el historial.*



Figura 2. *Inicio de una partida: Tona abajo, Rodri a la izquierda y Alejandro a la derecha; al centro, el mazo, el descarte, el color activo y el sentido del juego. Las cartas que no se pueden tirar se ven más oscuras.*

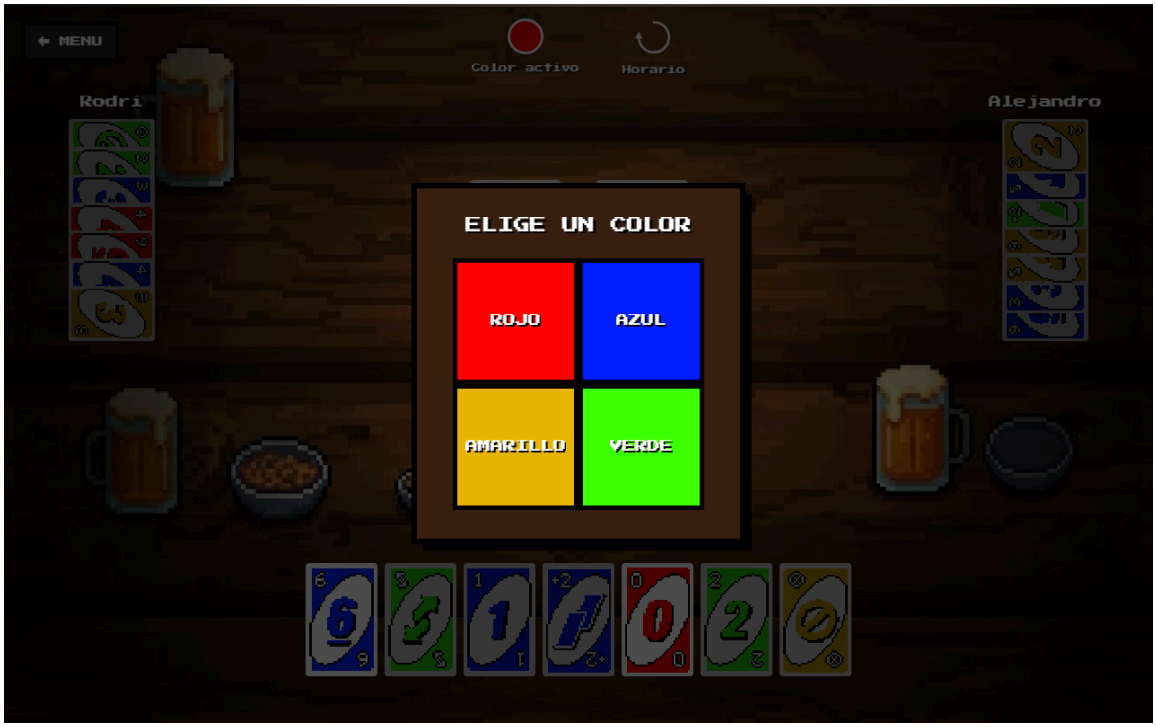


Figura 3. *Selector de color que aparece al tirar un comodín o un +4.*



Figura 4. *Botón ¡UNO! con la barra de 2 segundos después de tirar la penúltima carta.*



Figura 5. *Aviso de penalización cuando un jugador no canta UNO a tiempo.*

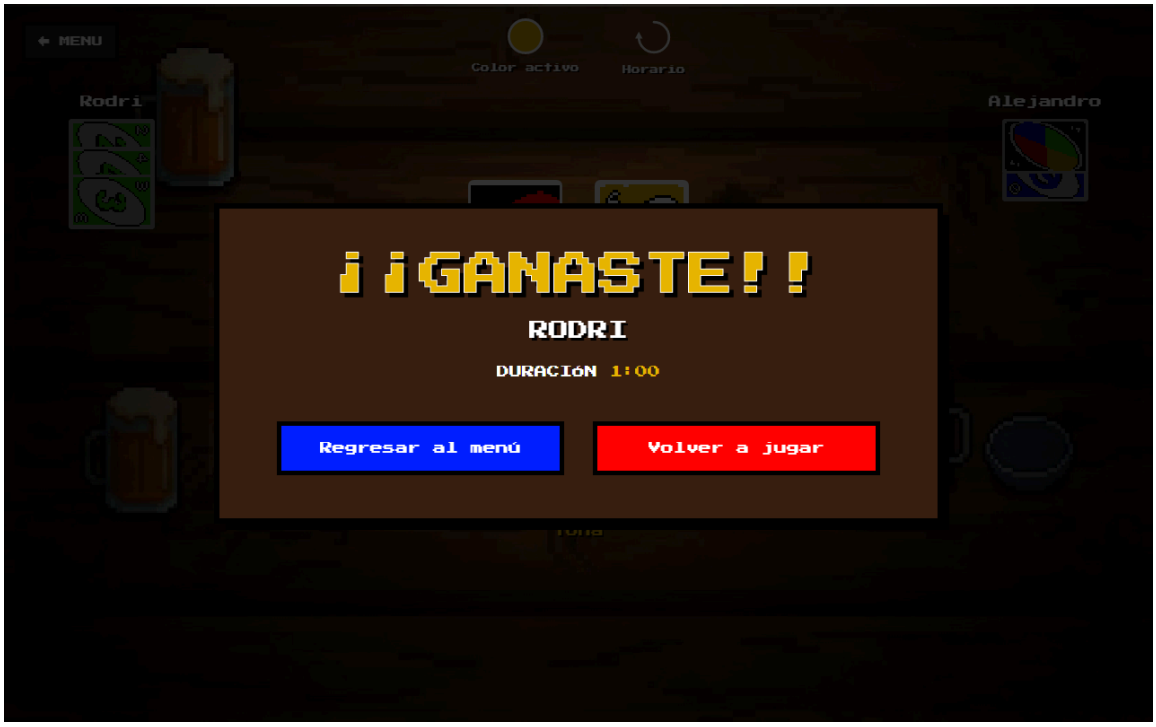


Figura 6. *Pantalla de victoria con el ganador y la duración de la partida.*



Figura 7. *Historial: tabla de clasificación, lista de partidas terminadas y bitácora con los movimientos de la partida elegida.*

2. División de tareas entre los integrantes del equipo

Cada integrante trabajó en su propio proyecto de la solución y en su propia rama de Git. Integramos los cambios a la rama main por medio de pull requests, después de revisarlos.

Integrante	Rol	Responsabilidades principales
Héctor Rodrigo Benítez Gama (Rodri)	Motor de juego (TheUno.Core)	Estructura de la solución, contratos compartidos, modelos, reglas oficiales, motor de juego y eventos.
Tonatiuh Charles Pérez Minssart (Tona)	Interfaz y experiencia de usuario (TheUno.UI)	Tablero, recursos gráficos, menú, música, conexión con el motor, diálogos, historial e integración final.
Alejandro Hernández	Base de datos y persistencia (TheUno.Infrastructure)	Diseño de la base de datos, migraciones, semilla de jugadores, repositorios, auditoría y API en Python.

2.1 Rodri: motor de juego (TheUno.Core y TheUno.Tests)

- **Estructura y Clean Architecture:** Creó la solución con los tres proyectos desacoplados (Core, Infrastructure y UI), asegurando un dominio puro sin dependencias externas, y configuró .gitattributes para compatibilidad multiplataforma (Windows y macOS).

- **Contratos compartidos:** Definió las interfaces (IGameEngine, IGameLogRepository, IMatchRepository) y DTOs (GameLogEntryDto, GameActionType), permitiendo el desarrollo paralelo e independiente de la interfaz y la persistencia.

- **Modelos de dominio:** Implementó Card (Guid único, visualización y puntaje oficial), Player (gestión de mano, estado de UNO y suma de puntos) y Deck (108 cartas, barajado Fisher-Yates, descarte y reciclaje conservando la carta activa).

- **Reglas oficiales (GameRules):** Validación de jugadas por color, valor o símbolo (IsPlayLegal), restricción oficial del comodín +4 evaluando la mano del jugador y bloqueo de acumulación de sanciones (+2 y +4).

- **Motor de juego (GameEngine):** Flujo de turnos circulares para 3 jugadores, inversión de sentido (1 y -1), efectos de cartas especiales (Skip, Reverse, +2, +4) y control de la regla de cantar UNO con penalización de 2 cartas por omisión.

- **Eventos desacoplados:** Diseñó el sistema de eventos (OnCardPlayed, OnCardDrawn, OnTurnChanged, OnUnoPenalty, OnGameOver) con sus clases de argumentos para notificar en tiempo real a la interfaz y al servicio de auditoría.

- **Pruebas unitarias (TheUno.Tests)**: Implementó la suite de pruebas automatizadas con xUnit y un repositorio en memoria (FakeGameLogRepository) para validar la baraja, el reciclaje, la rotación de turnos y las reglas sin depender de la UI ni de la base de datos.

2.2 Tona: interfaz y experiencia de usuario (TheUno.UI)

- **Tablero:** diseño de la mesa con los tres jugadores (las cartas de los lados giradas hacia cada jugador), el mazo y el descarte al centro, el indicador de color activo y la flecha del sentido del juego.
- **Recursos gráficos:** cartas en pixel art (8 bits) para las 54 caras y el reverso, fondos de taberna y mesa, la fuente Press Start 2P y un control propio para el texto con contorno.
- **Menú y música:** menú principal y una canción para el menú y otra para la partida, compatibles con Windows y macOS.
- **Conexión con el motor:** comandos para tirar carta, robar y pasar turno; suscripción a los eventos del motor para actualizar la mesa; cartas no jugables oscurecidas y manos de los demás jugadores deshabilitadas.
- **Diálogos:** selector de color para comodines, alerta para salir al menú, botón ¡UNO! con barra de 2 segundos, aviso de penalización y pantalla de victoria.
- **Historial:** pantalla con la tabla de clasificación, la lista de partidas terminadas y la bitácora de movimientos en español.
- **Integración final:** con el acuerdo del equipo, conectó la interfaz con la API y el motor: lectura del JSON de la API, endpoints para listar partidas y movimientos, y una versión del inicio de partida que usa los jugadores registrados en la base de datos. También escribió el README con los pasos para ejecutar el proyecto.

2.3 Alejandro: base de datos y persistencia (TheUno.Infrastructure)

- **Modelo de datos:** entidades Player, Match, MatchPlayer y GameLog, ajustadas para usar los mismos tipos de identificadores (Guid) que los contratos del Core.
- **Entity Framework Core:** UnoDbContext con la configuración de llaves y relaciones, las migraciones (InitialCreate y AddMatchPlayerAndUpdateContracts) y la semilla con los tres integrantes del equipo.
- **Repositorios:** MatchRepository (crear y cerrar partidas, tabla de clasificación con consultas LINQ) y GameLogRepository (guardar y consultar los movimientos de una partida).
- **Auditoría:** GameAuditService, que escucha los eventos del motor y los convierte en registros del log.
- **API en Python:** API REST con FastAPI y SQLite con los endpoints para crear y cerrar partidas, guardar movimientos y consultar las estadísticas de los jugadores, y los repositorios en C# que la consumen (ApiMatchRepository y ApiGameLogRepository).

- **Endpoints JSON**: Esquemas de la API

Endpoint	JSON

POST /matches	// Request
	{ “player_ids”: [“guid-1”, “guid-2”, “guid-3”] }
	// Response
	{ “matches_id”: “guid-partida”, “started_at”: “2026-10-11T13:54:00Z”}

POST /logs	{
	“match_id”: “guid-partida”,
	“turn_number”: 1,
	“player_id”: “guid-1”,
	“action”: “PlayerCard”,
	“card_detailis”: “Red 5”,
	“chosen_color”: “Red”,
	“timestamp”: “2026-10-11T13:54:05Z ”
	}

GET /player/stats	Resumen de estadísticas agregadas para el Leaderboard

- **Desacoplamiento de EF Core:** Se utilizó Entity Framework Core como herramienta inicial para el diseño del modelo de datos, generación de diagramas y la semilla predeterminada. Posteriormente, para cumplir con la conexión a una API, la persistencia activa se migró a un microservicio REST ligero con FastAPI en Python que gestiona de forma directa SQLite y se comunica con C# vía HttpClient.

3. Diagrama entidad-relación de la base de datos

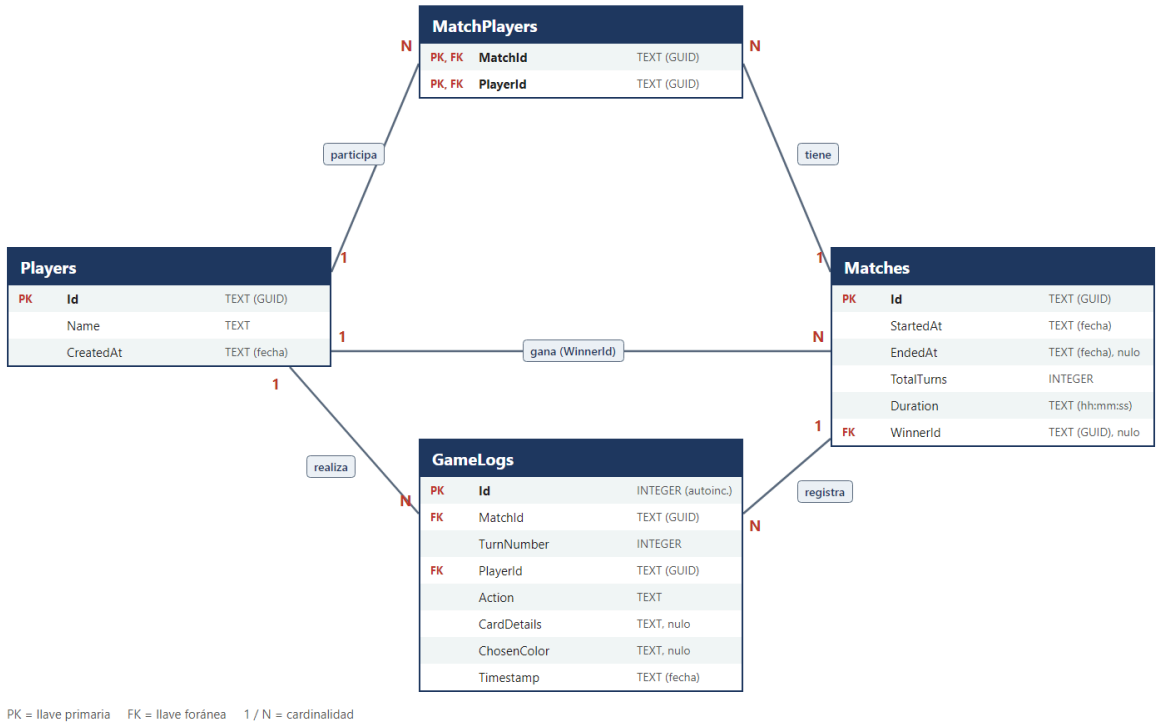


Figura 8. *Diagrama entidad-relación de la base de datos SQLite que usa la API.*

Tabla	Descripción
Players	Jugadores registrados. Contiene a los tres integrantes del equipo desde la semilla, por lo que no hace falta un formulario de registro.
Matches	Una fila por partida: inicio, fin, número de turnos, duración y ganador (WinnerId, llave foránea a Players). Una partida sin fecha de fin es una partida que no se terminó.
MatchPlayers	Relación muchos a muchos entre partidas y jugadores: qué jugadores participaron en cada partida. Su llave primaria es compuesta (MatchId, PlayerId).
GameLogs	Log del juego: cada movimiento de una partida con su número de turno, el jugador, la acción (tirar, robar, pasar, cantar UNO, penalización o fin de partida), la carta y el color elegido.

Relaciones: un jugador puede ganar muchas partidas (1:N); partidas y jugadores se relacionan de muchos a muchos por medio de MatchPlayers; y cada partida y cada jugador tienen muchos movimientos en GameLogs (1:N). Con estas tablas se obtiene el historial de partidas ganadas y perdidas por jugador y la traza completa de cada partida.

4. Listado de commits

Commits de la rama main del 21 de septiembre al 10 de octubre de 2026: 111 commits en total, incluidos los merges de los pull requests. Algunos integrantes aparecen con dos nombres de autor porque hicieron commits desde distintas computadoras o desde GitHub.

Integrante	Nombres de autor en Git	Commits
Rodri	Rodrigo Benitez	45
Tona	Soy Tonatiuh	45
Alejandro	Alejandro	21

Fecha	Autor	Commit	Mensaje
22/09/2026 11:33	Rodrigo Benitez	c4a0b	Normalizar saltos de linea con .gitattributes
22/09/2026 11:45	Soy Tonatiuh	0f66c	Prueba Inicial Push Branch
25/09/2026 00:18	Rodrigo Benitez	a2cb8	bases de la arquitectura del sistema con una sln con tres proj, Core, Infrastructure, UI
25/09/2026 18:43	Alejandro	aabbc	Se añadió a Infrastructure: clases {Player, Match, GameLog} como entidades, con sus respectivos atributos y propiedades
25/09/2026 19:25	Alejandro	23efd	Se añadió a infrastructure, {UnoDbContext] y se inicio La migración de EF Core, se generó la semilla principal con los tres jugadores ya predeterminados
27/09/2026 22:23	Rodrigo Benitez	3b965	feat: Implementacion de los contratos e interfaces para la infrastructura
28/09/2026 17:27	Alejandro	be338	feat(infrastructure): Guarde el trabajo para el pull
28/09/2026 17:27	Alejandro	a471a	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno into Infrastructure
28/09/2026 18:15	Alejandro	6f74e	feat(infrastructure): Implementa MatchRepository y GameLogRepository usando Core Contracts y Guid, se actualizo las entidades Player, Match Y GameLog porque tenian un formato diferente al implementado en Core
28/09/2026 19:08	Alejandro	dd9ec	feat: Agregue la carpeta de Services para implementar GameAuditService, esperando TheUno.Core.Engine
30/09/2026 20:44	Alejandro	14af3	Infrastructure, se añadió GameAuditService, aqui se gestiona el los eventos que suceden en una partida, lanzar carta, reversa, etcetera y se guardan en la base de datos
01/10/2026 11:37	Rodrigo Benitez	e5848f	Defincion de los modelos de Cartas y Jugador
01/10/2026 11:42	Rodrigo Benitez	bcef0	Se definen los posibles eventos de el juego
01/10/2026 11:55	Rodrigo Benitez	e1acb	feat:(core) Agrego las interfaces y contratos necesarios para la UI
01/10/2026 12:00	Héctor Rodrigo Benítez Gama	4472b	Merge pull request #1 from rodrigo-bg/Core
01/10/2026 12:04	Alejandro	02190	feat(infrastructure): Implemente las plnatillas para la api y app que conecta con la base de datos
01/10/2026 12:06	Alejandro	89825	Merge branch 'main' into Infrastructure
01/10/2026 20:21	Soy Tonatiuh	f06f4	FIX(UI): Resolver conflictos de merge con main y ajustar el proyecto de interfaz

Fecha	Autor	Commit	Mensaje
01/10/2026 20:32	Soy Tonatiuh	407dc	FEAT(UI): Agregar recursos graficos de las cartas al proyecto de interfaz
01/10/2026 20:47	Soy Tonatiuh	d86fa	FEAT(UI): Construir la estructura del tablero principal con los tres jugadores y la zona central
01/10/2026 21:17	Rodrigo Benitez	1c037	refactor: Modifiqué la clase Card para darle mas legibilidad
01/10/2026 21:22	Rodrigo Benitez	97fa2	feat(core): metodo para asignar valor a cartas y refactorizacion de GetDisplayText
01/10/2026 21:25	Soy Tonatiuh	33b32	FEAT(UI): Mostrar las manos y el estado de la mesa con datos de ejemplo
01/10/2026 21:25	Rodrigo Benitez	b6676	refactor(Player): implementacion de setters y getters explicitos
01/10/2026 21:44	Rodrigo Benitez	e2cc6	feat(core): agrega segundo constructor y calculo de puntos a Player.cs
01/10/2026 21:49	Rodrigo Benitez	6d7ed	feat(Models): agrega la clase Deck.cs e implementa el constructor
02/10/2026 11:06	Alejandro	83bd6	Merge branch 'Infrastructure' of https://github.com/rodrigo-bg/TheUno
02/10/2026 11:36	Rodrigo Benitez	26ed3	feat(Deck): agrega funciones getter de Deck.cs
02/10/2026 15:47	Soy Tonatiuh	2cff9	FEAT(UI): Cambiar el diseño de las cartas al estilo 8 bits en formato PNG
02/10/2026 16:16	Soy Tonatiuh	13069	FEAT(UI): Aplicar la tematica 8 bits al tablero con fondo, tipografia y paleta de colores
03/10/2026 15:26	Alejandro	3ed85	feat(infrastructure): Se creo la api para guardar y consultar datos, adema se refacrotizo GameAuditService por conflictos con los nombres de contracts y de los eventos
03/10/2026 16:00	Alejandro	f5b7a	fix(Infrastructure): Resolvi un problema con la migracion por el merge
03/10/2026 16:03	Alejandro	c6543	fix(Infrastructure): Resolvi un problema con la migracion por el merge, se arreglo problemas en GaeAuditService por nombres desiguales y ademas se implemento la api
04/10/2026 22:20	Rodrigo Benitez	132da	feat(deck): metodos de la clase Deck.csv (shufle, resset, draw, discard)
05/10/2026 19:41	Soy Tonatiuh	5a945	FEAT(UI): Agregar menu principal con navegacion al tablero y tipografia pixel en toda la interfaz
05/10/2026 19:59	Soy Tonatiuh	a4486	FEAT(UI): Agregar musica de fondo en loop durante el menu y la partida
05/10/2026 19:59	Soy Tonatiuh	02935	FIX(UI): Orientar las cartas del jugador 3 hacia su perspectiva
05/10/2026 20:11	Soy Tonatiuh	736c5	FIX(UI): Integrar los cambios de main con los modelos y contratos del Core
05/10/2026 20:15	Soy Tonatiuh	93f13	FEAT(UI): Conectar las cartas de la interfaz con el modelo Card del Core
05/10/2026 20:31	Soy Tonatiuh	e6ef6	FIX(UI): Reemplazar LibVLC para que la musica funcione en Windows y Mac
05/10/2026 22:52	Héctor Rodrigo Benítez Gama	5ed41	Merge pull request #2 from tonatiuh/Interfaz
05/10/2026 22:56	Rodrigo Benitez	97514	cambios antes del pull request
05/10/2026 23:00	Rodrigo Benitez	41cca	Merge branch 'main' into Core # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # # Lines starting with '#' will be ignored, and an empty message aborts # the commit.

Fecha	Autor	Commit	Mensaje
05/10/2026 23:08	Rodrigo Benitez	2aafc	feat(GameRules): Agreaga el metodo en la clase GameRules.cs el cual valida si la carta que se quiere tirar es valida
05/10/2026 23:13	Rodrigo Benitez	d50a2	feat(GameRules): Agrega metodo que determina si puede jugar un +4 o tiene al menos una jugada legal
05/10/2026 23:22	Rodrigo Benitez	219b3	feat(GameRules): Agrega metodo para evitar que algun jugador esté stackeando +2 o +4
06/10/2026 11:15	Alejandro	8db45	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno
06/10/2026 11:38	Rodrigo Benitez	aed7d	fix(Card): Se agrega un metodo que se esperaba en CardViewModel.cs
06/10/2026 11:39	Soy Tonatiuh	e8fef	FEAT(UI): Usar una cancion distinta para el menu y para la partida
06/10/2026 11:44	Héctor Rodrigo Benítez Gama	37750	Merge pull request #3 from rodrigo-bg/Core
06/10/2026 11:52	Alejandro	8a4d7	feat(infrastructure): Agregue la clase de MatchPlayer para guardar los jugadores de cada partida
06/10/2026 11:52	Alejandro	2e038	Merge branch 'main' of https://github.com/rodrigo-bg/TheUno
06/10/2026 11:53	Alejandro	fa290	Merge branch 'main' into Infrastructure # Please enter a commit message to explain why this merge is necessary, # especially if it merges an updated upstream into a topic branch. # # Lines starting with '#' will be ignored, and an empty message aborts # the commit.
06/10/2026 11:47	Soy Tonatiuh	43ce7	FIX(UI): Integrar los cambios del Core con las reglas y la baraja
06/10/2026 19:27	Soy Tonatiuh	ffb5e	FEAT(UI): Resaltar al jugador en turno y atenuar las manos de los demas
06/10/2026 19:46	Soy Tonatiuh	b6e68	FEAT(UI): Resaltar la carta al pasar el mouse sobre la mano del jugador en turno
06/10/2026 20:05	Soy Tonatiuh	48317	FEAT(UI): Agregar boton para regresar al menu con alerta de confirmacion
06/10/2026 20:05	Soy Tonatiuh	20501	FIX(UI): Quitar el marco del jugador en turno y dejar solo su nombre resaltado
06/10/2026 20:34	Soy Tonatiuh	b253c	FIX(UI): Iniciar la partida de ejemplo con el turno del Jugador 1
06/10/2026 21:00	Soy Tonatiuh	b7b07	FEAT(UI): Agregar selector de color para comodines
07/10/2026 12:50	Soy Tonatiuh	5b983	FEAT(UI): Agregar boton para cantar UNO frente a cada jugador
07/10/2026 16:09	Soy Tonatiuh	66b36	FEAT(UI): Agregar pantalla de victoria
07/10/2026 16:38	Soy Tonatiuh	26fcb	FIX(UI): Quitar la fuente Inter que no se usaba y fijar el tema oscuro
07/10/2026 16:44	Soy Tonatiuh	6750d	FIX(UI): Usar CardColor para el color activo
07/10/2026 17:16	Soy Tonatiuh	0d75c	FIX(UI): Ordenar BoardViewModel
07/10/2026 18:44	Rodrigo Benitez	213b8	feat(GameEngine): se implementan los atributos de la clase GameEngine
07/10/2026 18:47	Rodrigo Benitez	2f999	feat(GameEngine): Implmenta getters de GameEngine
07/10/2026 19:11	Rodrigo Benitez	fbe68	feat(GameEngine): Implementa el metodo StartGame que limpia y comienza la partida
07/10/2026 19:16	Rodrigo Benitez	2417f	feat(GameEngine): agrega todas las funciones al metodo StartGame
07/10/2026 19:25	Rodrigo Benitez	67e3e	feat(GameEngine): se agregan los metodos para saber si una carta puede ser jugada y el metodo para ejecutar la carta
07/10/2026 19:50	Rodrigo Benitez	3e7b8	feat(GameEngine): metodos para robar, pasar turno y cantar UNO

Fecha	Autor	Commit	Mensaje
07/10/2026 20:10	Rodrigo Benitez	9f81e	feat(GameEngine): agrega metodos privados que ya eran usados por los metodos publicos
07/10/2026 20:14	Rodrigo Benitez	cdd98	feat(GameEngine): agrega metodos para repartir cartas a los jugadores y revisar si el jugador tiene penalizaciones(si no cantó UNO)
07/10/2026 20:30	Rodrigo Benitez	5564c	feat(GameEngine): agrega funciones para monitoreo de jugadas y para terminar la partida
07/10/2026 20:34	Rodrigo Benitez	db064	fix(Player.cs): Agrega metodos faltantes en la clase Player.cs
07/10/2026 23:49	Alejandro	148df	feat(infrastructure): Añadi la entidad de MatchPlayers para guardar los jugadores de una cierta partida, esto trajo cambios a la match. Agregue ApiGameLog y ApiMatch repositorios, estos para recibir los datos de la api, recibe el json y los convierte para su uso. Se hizo un cambio en MatchRepository para guardar los id de los jugadores. En GameAuditService se modifico para que cada partida nueva se reinicie la suscripcion a los eventos. Se agrego la configuracion de la matchplayers a UnoDbContext. Se agrego la conexion de la api a la app principal en App.axaml.cs
08/10/2026 19:46	Rodrigo Benitez	52d20	feat(Events): Agrega la funcion notificadora de la carta jugada
08/10/2026 19:49	Rodrigo Benitez	8b97f	fix(Events): Elimina un archivo obsoleto que funcionó para declarar la arquitectura
08/10/2026 19:52	Rodrigo Benitez	144e7	feat(Events): implementa las clases previamente declaradas en GameEvents
08/10/2026 20:05	Rodrigo Benitez	7972a	fix(Events): Por coherencia con la arquitectura separa IGameLogRepository.cs en 3 clases. una interfaz, un enum y un DTO
08/10/2026 20:08	Rodrigo Benitez	8d638	feat(GameEngine): Al terminar una partida hace un logaction hacia la BD
08/10/2026 20:12	Héctor Rodrigo Benítez Gama	2f4f6	Merge pull request #4 from rodrigo-bg/Interfaz
08/10/2026 20:15	Héctor Rodrigo Benítez Gama	2cd15	Merge branch 'main' into Infrastructure
08/10/2026 20:18	Héctor Rodrigo Benítez Gama	02cbd	Merge pull request #5 from rodrigo-bg/Infrastructure
08/10/2026 20:26	Rodrigo Benitez	7cef5	Merge branch 'main' into Core
08/10/2026 20:40	Rodrigo Benitez	bf0f9	fix(Core): arregla algunas discrepancias con Infrastructure en el nombramiento de las variables de GameLogEntryDto.cs y metodos faltantes
08/10/2026 20:45	Héctor Rodrigo Benítez Gama	8d590	Merge pull request #6 from rodrigo-bg/Core
09/10/2026 10:22	Soy Tonatiuh	2ecb7	FEAT(UI): Iniciar la partida con el motor de juego
09/10/2026 10:40	Soy Tonatiuh	98303	FEAT(UI): Tirar cartas con un clic
09/10/2026 11:01	Soy Tonatiuh	49b2d	FEAT(UI): Robar del mazo y pasar turno
09/10/2026 11:45	Soy Tonatiuh	11fcd	FEAT(UI): Oscurecer las cartas que no se pueden tirar
09/10/2026 11:48	Soy Tonatiuh	98a4e	FIX(UI): Rediseño de cartas 8 bit
09/10/2026 11:54	Alejandro	a9f79	fix(infrastructure): Cambios minimos para MatchPlayer
09/10/2026 11:56	Alejandro	a9bb0	Merge branch 'main' into Infrastructure Union de UI y Core
09/10/2026 12:03	Soy Tonatiuh	bbd83	FIX(UI): Avisar al motor cuando un jugador canta UNO

Fecha	Autor	Commit	Mensaje
09/10/2026 12:09	Soy Tonatiuh	e8a97	FEAT(UI): Mostrar la pantalla de victoria al terminar la partida
09/10/2026 12:18	Soy Tonatiuh	b8532	FEAT(UI): Dar 2 segundos para cantar UNO despues de tirar la penultima carta
09/10/2026 12:30	Soy Tonatiuh	52cbc	FEAT(UI): Avisar la penalizacion por no cantar UNO
09/10/2026 12:36	Soy Tonatiuh	8eb5a	FEAT(UI): Agregar pantalla de historial con datos de ejemplo
09/10/2026 12:42	Soy Tonatiuh	4e5bc	FEAT(UI): Abrir el historial desde el menu
09/10/2026 12:54	Soy Tonatiuh	7ddfa	FIX(UI): Recibir el motor de juego en el constructor de BoardViewModel
09/10/2026 12:55	Soy Tonatiuh	e5fb4	FEAT(UI): Mostrar duracion y puntos en la pantalla de victoria
09/10/2026 15:21	Héctor Rodrigo Benítez Gama	75ff0	Merge pull request #10 from rodrigo-bg/Interfaz
10/10/2026 00:46	Alejandro	3107c	fix(infrastructure): Se aseguro que la claves primarias existan en MatchId y PlayerId. Se configuro para que se cargue la semilla correctamente. Se uso solo GameEngine para el registro de auditoria y evitar duplicados. Se amplio PlayerStatsDto para guardar duración promedio y puntajes de partida. Se realiza el registro completo de las partidas(cartas jugadas, turnos, penalizaciones. ganador, duración)
10/10/2026 10:03	Soy_Tonatiuh	764af	Merge pull request #11 from rodrigo-bg/Infrastructure
10/10/2026 10:24	Soy Tonatiuh	6e524	FIX(UI): Ajustar el historial al nuevo formato de estadisticas
10/10/2026 10:42	Soy Tonatiuh	8b632	FEAT(UI): Conectar el motor y la API para guardar las partidas
10/10/2026 11:58	Soy Tonatiuh	87751	FEAT(UI): Guardar las partidas en la API y mostrar el historial real
10/10/2026 12:51	Soy Tonatiuh	96a89	FEAT(UI): Mostrar la lista de partidas y la bitacora de movimientos en el historial
10/10/2026 14:42	Soy Tonatiuh	c0066	FEAT(UI): Agregar README con los pasos para ejecutar el juego y la API
10/10/2026 21:29	Héctor Rodrigo Benítez Gama	2ba63	Merge pull request #12 from rodrigo-bg/Interfaz

5. Listado de ramas generadas

Trabajamos con una rama por integrante y la rama main como rama principal. Cada rama se integró a main por medio de pull requests; además, cada integrante traía los cambios de main a su rama antes de subir los suyos para resolver los conflictos de su lado.

Rama	Responsable	Uso
main	Todo el equipo	Rama principal; recibe los cambios de las demás ramas por pull request.
Core	Rodri	Motor de juego (TheUno.Core). Se eliminó del repositorio después de integrarse a main.
Infrastructure	Alejandro	Base de datos, repositorios y API (TheUno.Infrastructure).
Interfaz	Tona	Interfaz gráfica (TheUno.UI) e integración final.

6. Problemas encontrados y cómo los solucionamos

6.1 Sobre el proyecto

Motor de juego (Rodri)

- **Discrepancia de contratos y argumentos nombrados:** Al integrar la persistencia, GameAuditService fallaba por argumentos en PascalCase en GameLogEntryDto. Se ajustó el constructor del DTO para admitir parámetros nombrados y posicionales sin romper el código de infraestructura.

- **Incompatibilidad de nombres en eventos:** La capa de infraestructura requería propiedades con nombres alternativos (EffectiveColor y DrawnCard). Se agregaron propiedades alias en CardPlayedEventArgs y CardDrawnEventArgs para garantizar compatibilidad hacia atrás.

- **Manejo de archivos swap y bloqueos en Git:** Durante una fusión en macOS (git merge main), un archivo temporal residual (.MERGE_MSG.swp) bloqueó la confirmación en Vim. Se eliminó el archivo swap residual y se completó la confirmación de la fusión sin contratiempos.

- **Aislamiento del dominio frente a la interfaz:** Se mantuvo el motor agnóstico a la interacción física sin programar bots en el Core, delegando el flujo de turnos en modalidad local (hot-seat) y el control de clics exclusivamente a los ViewModels de Avalonia.

- **Discrepancias de firmas en pruebas unitarias:** Se adaptó la suite de xUnit para tratar GameRules como clase instanciable y proveer la mano completa del jugador en IsPlayLegal, validando con exactitud la restricción oficial del comodín +4.

Interfaz (Tona)

- **Música en Windows y macOS:** la primera versión usaba LibVLC y no funcionaba en macOS. La cambiamos por NAudio en Windows y el comando afplay en macOS.
- **Cartas borrosas:** Avalonia suavizaba las imágenes en pixel art. Desactivamos la interpolación con RenderOptions.BitmapInterpolationMode, que tiene que ir en el elemento raíz de la vista.
- **Cartas laterales recortadas:** las manos de los lados están giradas y el control que las gira recorta lo que sale de la carta. El efecto al pasar el mouse se aplicó al contenedor de cada carta.

- **Cantar UNO:** el botón aparece cuando el jugador ya tiró su penúltima carta y el turno es del siguiente, así que el método del motor para cantar UNO no le aplicaba. La interfaz da 2 segundos, marca al jugador que cantó y guarda el movimiento en el log.
- **Partida doble al iniciar:** con la API cerrada la conexión tardaba en fallar y un segundo clic iniciaba otra partida. Bloqueamos los clics y mostramos "CARGANDO..." mientras se crea la partida.

Base de datos y API (Alejandro)

- **Tipos de identificadores distintos:** las primeras entidades no coincidían con los contratos del Core. Se actualizaron para usar Guid como identificador.
- **Migración rota después de un merge:** al integrar los cambios de main, la migración de Entity Framework dejó de coincidir con el modelo. Se corrigió y se regeneró.
- **Llave de MatchPlayer:** la tabla de jugadores por partida no tenía llave primaria y la base de datos no se podía crear. Se definió una llave compuesta (MatchId, PlayerId).
- **La API y C# no se entendían:** la API responde en snake_case (match_id) y C# esperaba MatchId, así que el Id de la partida llegaba vacío. Se configuró la lectura con JsonNamingPolicy.SnakeCaseLower.
- **Movimientos rechazados por la base:** el motor creaba jugadores con identificadores nuevos en cada partida y la base los rechazaba por las llaves foráneas. Ahora la partida inicia con los jugadores registrados y con el Id de la partida creada en la API.
- **Auditoría duplicada:** el motor y GameAuditService registraban los mismos movimientos. Se dejó solo el registro del motor.

6.2 Sobre el uso de Git y GitHub

- **Saltos de línea entre sistemas:** trabajamos desde Windows y macOS, y los saltos de línea distintos generaban cambios falsos. Los normalizamos con un archivo .gitattributes.
- **Mantener las ramas al día:** para traer el trabajo de los demás usamos el flujo git switch main, git pull origin main, git switch a la rama propia y git merge main antes de cada push.
- **Conflictos al integrar:** al unir las ramas surgieron conflictos, sobre todo en las migraciones de la base de datos y en los nombres de los contratos. Los resolvimos en la rama de cada integrante antes de abrir el pull request.
- **Pull request que rompió la compilación:** un cambio en un contrato del Core llegó a main y la interfaz dejó de compilar. Lo corregimos con un commit de ajuste en la rama Interfaz.

- **Coordinación del trabajo mediante GitHub Issues:** Al pasar a la etapa de conexión entre el motor y la interfaz, existía el riesgo de colisiones entre colaboradores que editaban los mismos archivos de la vista. Para evitarlo, se desglosó el trabajo pendiente en tres issues técnicos independientes en GitHub con títulos descriptivos y criterios de aceptación específicos. Esto permitió establecer un orden secuencial claro de entrega sin bloqueos mutuos en las ramas activas.
- **Cambios que se colaban en los commits:** al editar archivos mientras se preparaba un commit, git add . incluía cambios que no correspondían. Empezamos a revisar el diff completo antes de cada commit.
- **Commits demasiado grandes:** un refactor de muchos archivos en un solo commit era difícil de revisar y se revirtió. Trabajamos con commits pequeños y mensajes con prefijos como feat y fix.
- **Archivos que no debían subirse:** se subieron archivos de herramientas, caché de Python (__pycache__) y bases de datos locales (.db). Los quitamos del repositorio y los agregamos al .gitignore.
- **Ordenar el historial:** usamos git stash para separar cambios de un mismo archivo en commits distintos y git rebase para juntar commits antes de subirlos.

7. Conclusión

Desarrollamos un juego de UNO completo y gráfico en C# que sigue las reglas oficiales y que guarda en una base de datos a los jugadores, el historial de partidas ganadas y perdidas y el log con todos los movimientos de cada partida. Separar el proyecto en motor, interfaz y persistencia, con contratos definidos desde el inicio, nos permitió trabajar en paralelo: la interfaz se pudo diseñar con datos de ejemplo mientras el motor y la base de datos estaban en desarrollo, y después solo hubo que conectarlos.

La parte más difícil fue la integración. Los problemas más importantes aparecieron cuando juntamos el trabajo de los tres: nombres distintos en los contratos, identificadores que no coincidían entre el motor y la base de datos, y una API que respondía en un formato diferente al que esperaba C#. Resolverlos nos enseñó la importancia de acordar los contratos con detalle, probar la solución completa antes de unir un pull request y comunicar los cambios que afectan a los demás.

En cuanto a Git y GitHub, trabajar con una rama por integrante y pull requests nos ayudó a revisar los cambios antes de integrarlos a main, y aprendimos a mantener nuestras ramas al día, resolver conflictos y hacer commits pequeños y claros. Como mejoras futuras quedan permitir que varias computadoras jueguen en red, guardar los puntos reales de cada partida.